



Building a Finance Domain RAG AI Assistant with LangChain: Step-by-Step Guide

Project Overview and Objectives

This project will guide you through creating an end-to-end **Retrieval-Augmented Generation (RAG)** system focused on finance. The goal is to build a question-answering assistant that can **answer finance-related questions** using up-to-date sources (e.g. SEC 10-K/10-Q filings, earnings call transcripts, financial news), while providing **source citations** for transparency. We will integrate all core LangChain components – chains, agents, memory, tools, retrieval, prompt templates, and callbacks – into a cohesive pipeline. The final system will be deployed on AWS using cost-effective services (AWS Lambda, EC2, S3, API Gateway, and optionally SageMaker) with an easy-to-use web UI. Throughout the project, we'll emphasize **clear, step-by-step instructions** and modern MLOps best practices (CI/CD, versioning, monitoring). By the end, a single Python developer (you) with basic AWS experience will have mastered building and deploying an AI solution that meets real-world **financial analytics** needs.

Key Features of the Solution:

- **Finance Domain Focus:** The assistant is tailored to financial documents and queries, providing accurate, concise answers about companies' financials and news, without giving investment advice or venturing off-topic.
- **Retrieval-Augmented Generation:** The system retrieves relevant snippets from a custom knowledge base (filings, transcripts, etc.) and feeds them into an open-source Large Language Model (LLM) to ground its answers in reality. This minimizes hallucinations and keeps answers up-to-date ¹ ².
- **Source Attribution:** Every answer cites its sources (with document names or links), increasing user trust and supporting fact-checking ³. Users can see exactly where the information came from.
- **Full LangChain Integration:** We will use LangChain to orchestrate the pipeline – loading and chunking documents, creating embeddings for retrieval, using chains for Q&A, employing **memory** for context awareness, and even adding an **agent with tools** for advanced tasks.
- **Open-Source Models Only:** All AI models (LLMs and embedding models) will be open-source (e.g. Llama 2, Falcon, Mistral, or similar). This avoids API costs and restrictions, and allows local or AWS-hosted deployment. Modern open models are powerful enough for this task – e.g. Mistral 7B can even outperform Llama 2 13B on many benchmarks ⁴. We'll choose models that balance quality with speed and cost, and use open-source embedding models for semantic search.
- **AWS Deployment (Cost-Effective):** The entire app will be deployable on AWS on a tight budget. We'll use **Amazon S3** for storing data (and possibly hosting a static web front-end), **AWS Lambda** and/or a small **EC2 instance** for the backend logic, **API Gateway** to provide secure API endpoints, and optionally **Amazon SageMaker** for hosting the LLM if needed (e.g. using a managed endpoint for the model). We'll discuss trade-offs between these options to keep costs low.
- **Modern MLOps Practices:** We incorporate **CI/CD** for our code and infrastructure (so updates can be deployed reliably), **model and data versioning** (so we know which model/checkpoint or document version is in use), a strategy for **continuous improvement** (updating the knowledge base with new filings or retraining as needed), and **logging/monitoring** via AWS CloudWatch to track usage, performance, and catch errors in production. This ensures the system remains **maintainable and reliable** over time.
- **Developer Learning Focus:** This project touches on all key AI engineering skills. You'll simulate **stakeholder collaboration** by defining requirements with finance domain experts, show **technical**

leadership in making design decisions (like choosing a modular, open-source approach for flexibility ⁵ ⁶), perform **AI integration** by embedding the solution into a web app and potentially an existing business workflow, plan for **monitoring and maintenance** to keep the system running smoothly, and maintain a sharp **financial application focus** to ensure the AI's output is relevant and compliant (e.g. no hallucinated numbers or improper financial advice).

In summary, we'll build a financial Q&A chatbot that is *accurate*, *transparent*, and *maintainable*. Let's start with the planning and then walk through implementation step by step.

Requirements and Planning (Stakeholder Collaboration)

Every successful AI project starts with understanding the **requirements and constraints**, ideally in collaboration with stakeholders. In a real scenario, you (as the AI Engineer) would talk to financial analysts, business users, or product managers to clarify what they need from the assistant. For this project, we'll assume the following requirements based on stakeholder input:

- **Use Case:** The assistant will answer questions about public companies' financials and recent developments. Example queries might include:
 - "What was **ACME Corp's revenue in 2022** according to their filings?"
 - "Did **ACME's CEO** mention **supply chain issues** in the last earnings call?"
 - "Summarize the **latest news** about ACME Corp from this quarter."
- **Scope of Knowledge:** The system should focus **only on finance topics** – primarily information from official documents (10-K, 10-Q, annual reports) and credible news sources. It **must not give personal financial advice** or venture into unsupported topics (this avoids legal and ethical issues ⁷). Stakeholders want professional, factual answers, not speculation.
- **Accuracy and Compliance:** Because this is a financial domain, accuracy is paramount. Any figures or statements must come from the approved sources. Stakeholders are concerned about LLMs "making things up," so we will ensure all answers are backed by retrieved documents (reducing hallucination risk ⁸ ⁹) and include citations. We will also instruct the model to **avoid opinions or recommendations** (no "Buy this stock" advice).
- **User Interaction:** The assistant should be accessible through a simple web interface where users can type a question and get an answer with references. It should support follow-up questions in a conversation (contextual Q&A), at least within a single session. This means we need some form of **memory** to handle context from prior queries.
- **Performance:** Users would like answers within a few seconds. We aim to optimize for low latency – choosing efficient models and retrieval methods – but given limited resources, there may be some trade-off (e.g. slightly slower answers for complex queries). We assume moderate usage (a single analyst or a small team using it interactively), so we don't need to handle massive traffic, but the design should allow scaling up if needed.
- **Cost Constraints:** Stakeholders have a limited budget for this prototype. We will use **free or low-cost AWS tiers** where possible and **open-source models** to avoid licensing fees. We'll design the system to scale down to zero cost when idle (for example, using serverless components that incur costs only on usage, and choosing minimal-size instances).
- **Security & Privacy:** All data we use (financial filings, transcripts, news) is public, so privacy concerns are minimal. However, if deployed in a company setting, we would ensure the solution is **secure** – e.g., restricting the API to authorized users and keeping data in a private environment. For our

guide, we'll mention best practices like using API keys or AWS IAM roles to control access, but will not delve into deep security setup.

By clarifying these needs, we align our solution with stakeholder expectations – a crucial skill in **stakeholder collaboration**. Next, we'll translate these requirements into a system design.

System Architecture Overview (Design & Technical Leadership)

Now that we know what we need, let's design the architecture of the RAG system. At a high level, the solution has two major stages: **(1) offline data preparation (indexing)**, and **(2) online query handling (retrieval + generation)** ¹⁰. We will also map these stages onto AWS components for deployment.

Architecture Highlights: - We'll maintain a **knowledge base** of financial documents, stored as text embeddings in a **vector database** for fast similarity search. - When a user asks a question, the system will **retrieve relevant document chunks** from the vector DB, then feed them into an **LLM** which generates the answer. - The entire flow will be coordinated by our LangChain application, which can be hosted on AWS (using Lambda or EC2). A simple web UI will communicate with this backend via API calls.

Figure 1: High-level architecture for the RAG system. The solution is divided into modules – data ingestion, vector store, LLM (model) serving, orchestration, and user interface ¹¹ ¹². Documents (10-Ks, 10-Qs, etc.) are stored in S3, processed into embeddings (via Lambda or a batch job) and indexed in a vector store. At query time, a user's question flows through API Gateway to a Lambda function (or server) that retrieves relevant vectors and then calls the LLM (hosted on SageMaker or EC2) to generate an answer, which is returned to the user.

A few notes on the AWS services illustrated in the architecture: - **Amazon S3:** Serves as durable storage for our data. We will keep the raw documents here (e.g. PDFs or text of filings) and possibly the serialized vector index or embedding files. S3 will also host static files for the frontend UI. - **AWS Lambda:** Used for serverless compute. We can use Lambda in two ways: (1) to run the **ingestion pipeline** that reads documents from S3, chunk and embed them, and store into the vector DB (as shown in Figure 1, step 1) ¹³; and (2) to handle **user queries** by retrieving relevant info and calling the LLM (Figure 2 steps) ¹⁴. Lambda is cost-effective because it only runs when invoked and scales automatically. However, Lambda has memory and time limits, so we must ensure our model inference can work within those (we'll discuss strategies soon). - **Vector Store (Chroma/FAISS/OpenSearch):** This is where we'll store embeddings for efficient similarity search. We have flexibility here – we could use an in-memory store like **ChromaDB** or **FAISS** (simple and open-source), or a managed service like **Amazon OpenSearch** with its kNN vector index. Given our scale (a few hundred or thousand documents) and cost goals, an open-source solution we manage ourselves is ideal ⁵. For example, we might embed documents and keep the vectors in a local FAISS index file on disk or in memory. - **LLM Serving (EC2 or SageMaker):** The core LLM (like Llama-2 or Mistral) can be hosted in two main ways: - **EC2:** Launch a compute instance (with enough CPU/RAM, or a GPU if available) and run the model there (e.g. via a Flask API or using a library like Hugging Face Transformers). You have full control here and can choose a smaller instance to save cost. You'd need to manage the server (starting/stopping it). - **SageMaker Endpoint:** Use Amazon SageMaker to deploy the model as a managed service. SageMaker can host popular open-source models (it even has JumpStart templates). This is convenient since AWS will handle loading the model and scaling, but it could be costlier if left on 24/7. SageMaker does offer **Serverless Inference** for intermittent usage or you can use a smaller instance and shut it down off-hours. - *Which to choose?* For this guide, we'll illustrate using a SageMaker endpoint for the LLM (to show a fully

managed pipeline). But we will also mention how you could instead use an EC2 instance if you prefer to avoid SageMaker. Both approaches align with a modular architecture – you could swap one for the other with minimal changes ¹⁵ ⁶ . - **API Gateway**: Acts as the entry point for the front-end to call our backend. It will expose a REST API (or WebSocket if we wanted streaming, but we can keep it simple with REST). API Gateway will invoke the Lambda function for queries. This adds a small cost per call but allows authentication, monitoring, and scaling easily. Alternatively, if using an EC2 with a public API, API Gateway could forward requests to it or we might skip API Gateway entirely in that case. We'll use it in our main design to keep the front-end decoupled from backend implementation. - **Front-End UI**: A simple web page (HTML/JavaScript) will allow the user to input questions and display answers. To keep costs down, we can host this as a **static website on S3** (which can be made public or behind authentication). The JS will call the API Gateway endpoint to get answers. We'll ensure the UI displays the answer text with citations and perhaps source links. (Alternatively, during development, one could use a quick UI framework like Streamlit or Gradio on localhost to test the bot, but for deployment, a static web page is simplest and nearly free.)

In terms of **data flow**, here's what happens step-by-step when everything is up and running (the online query path):

Figure 2: Retrieval-Augmented Generation query workflow. When the user submits a question (step 1), the system converts it into an embedding (step 2) and searches the vector store for relevant content (step 3). The top-matching snippets are fetched (step 4) and combined with the user query to prompt the LLM (step 5). The LLM, using both the query and retrieved context, generates a grounded answer which is then returned to the user (step 6) ¹⁴ ¹⁶ . This approach ensures the LLM's response stays factual and is derived from the provided documents rather than the model's training data alone.

By designing the system in modular components (data ingestion, retrieval, generation, etc.), we make it **flexible** and easy to upgrade. For example, we can swap out the vector database or the LLM model in the future without overhauling the whole pipeline ⁵ ¹⁷ . As the **technical lead** of this project, you should communicate these design choices and their rationale to stakeholders and team members. Emphasize how this architecture meets their needs: it's **accurate** (via RAG), **transparent** (with sources), **scalable & cost-efficient** (serverless components), and **adaptable** (open-source, modular design).

With the architecture in mind, let's proceed to set up the components step by step, starting with the data.

Data Sources and Ingestion Pipeline (Data Management)

The first step is to gather and preprocess the data that will feed our RAG system. In the finance domain, relevant data sources include: - **SEC Filings (10-K, 10-Q, Annual Reports)**: These are rich with official financial figures and company discussions. We can obtain them from the SEC's EDGAR database ¹⁸ (which provides downloads for public filings). For example, we might download recent 10-K and 10-Q reports for the companies of interest (in HTML, PDF or text format). - **Earnings Call Transcripts**: Many companies have quarterly earnings calls with analysts; transcripts are often available via investor relations sites or finance news sites. These contain qualitative insights (e.g. CEO's comments) that complement the raw numbers in filings. - **Financial News Articles**: To capture the latest developments (e.g. major product announcements, market news), we can include recent news. Good sources might be press releases or reputable financial news outlets. For simplicity, we might manually collect a few example articles or use an API/RSS feed for the most recent news on the target companies.

Data ingestion involves converting these raw texts into a form our system can use (namely, embedding vectors stored in a database). Here's how to do it:

1. **Gather Documents:** Download or collect the documents. You could script this (e.g. using SEC EDGAR's RSS feeds or API to get filings, or scraping a website for transcripts), or start with a small curated set (for a prototype). Ensure each document is saved in a readable text format (e.g. plain text or Markdown). If you have PDFs, you can use a PDF parser to extract text.
2. **Document Loading:** Use LangChain's **Document Loaders** or your own code to load each document's text into memory. In LangChain, a `Document` typically has `.page_content` (the text) and `.metadata` (like title, date, source). We'll populate metadata with useful info (e.g. filename, document type, date).
3. **Text Splitting:** Financial reports can be very large (tens or hundreds of pages). Feeding a whole report into an LLM is impractical, so we need to **split documents into chunks**. We use LangChain's **TextSplitter** (e.g. `RecursiveCharacterTextSplitter`). We might split by paragraph or by a maximum token length (like 500 tokens per chunk) ¹⁹. The goal is to have chunks that are semantically coherent and not too long for the model's context window. For example, an 80-page 10-K might split into 300 chunks of a few paragraphs each.
4. **Embeddings Model:** Choose an open-source **embedding model** to convert text chunks into vector representations. Embeddings map text into a high-dimensional space such that similar texts are near each other. Since we're avoiding proprietary models, we can use a **Hugging Face** embedding model (for example, `sentence-transformers/all-MiniLM-L6-v2` is a popular lightweight option for semantic search). There are also larger models like `intfloat/e5-large-v2` or domain-specific ones, but a smaller model is faster and may be sufficient for our Q&A needs. Each chunk of text will be passed to the embedding model to get a vector (say 384 dimensions for MiniLM).
5. **Vector Store Indexing:** Use LangChain's integration with vector stores to store these embeddings. For instance, we can use **ChromaDB** or a simple FAISS index via LangChain's `FAISS` vector store wrapper. Each stored vector is associated with the chunk text and its metadata. After this step, we have an index that can be queried by vector similarity. (Under the hood, vector stores typically use cosine similarity or dot product to find nearest neighbors of a query vector.)
6. **Storage of the Index:** Since we want to deploy on AWS, we need to consider where this index lives. If using a managed service (like OpenSearch or Pinecone, etc.), it's hosted by them. For an open-source solution on AWS, we have a couple of options:
7. **On Lambda:** Lambda has ephemeral storage (up to 10 GB in a temp `/tmp` directory). We could package the index with our function or load it from S3 each time. If the index is not too large, this can work, but cold-starts will be longer when loading the index. We might also use an Amazon EFS (file system) mounted to Lambda for persistent storage if needed.
8. **On EC2 or SageMaker:** If we run the LangChain retrieval component on EC2 or in the SageMaker container, we can store the index on the instance's disk. For example, we might keep a FAISS index file on an EC2 volume or SageMaker's storage, and load it at startup.
9. **DynamoDB or OpenSearch:** As an alternative, one could use DynamoDB with the new vector search capability or Amazon OpenSearch Serverless for persistence. However, these might incur extra cost and complexity. For our minimal solution, we'll assume using a local vector store (Chroma or FAISS) either in-memory or loaded from S3.

By the end of ingestion, we'll have a populated vector store of embeddings. In summary: - Let's say we processed 50 documents (filings, transcripts, etc.) which resulted in ~1000 text chunks. We embedded each and stored them. We also saved mapping info so we can retrieve the original doc name or URL for citations. - This "indexing" step can be run offline (e.g. in a Jupyter notebook or a one-time script). In production, you

might automate it – for example, set up a **Lambda function triggered whenever a new file is added to S3** to update the index, keeping the knowledge base current. This ensures the system stays up-to-date with new filings or news (a crucial need for finance) ²⁰ .

Implementation Tip: Use LangChain's high-level APIs like `Langchain's TextLoader` for loading, `RecursiveCharacterTextSplitter` for splitting, and `Chroma.from_documents()` or `FAISS.from_documents()` to create the index in a few lines of code. This will internally handle embedding each chunk with your chosen model. Keep an eye on AWS resource limits if doing this in Lambda – for large initial indexing, it might be easier to do it locally or on an EC2 notebook instance and then upload the index.

Now that the data is ingested and indexed, let's choose our models and set up the Q&A pipeline.

Choosing the LLM and Embeddings (Open-Source Model Selection)

Choosing the right models is a critical step. We need: 1. **An LLM** to generate answers (and possibly to function as an agent). 2. **An Embedding Model** to vectorize text for retrieval.

LLM (Answer Generation Model): We will use an **open-source LLM** both for cost reasons and flexibility. Some viable options in 2025 include: - **LLaMA 2 (Meta)** – available in 7B, 13B, 70B parameter sizes. The 13B version can give fairly good answers and might just fit on a single GPU (or can be quantized for CPU). LLaMA 2's fine-tuned chat model is well-behaved for Q&A. *License:* Free for research/commercial use with some conditions (it's not pure Apache 2.0, but generally usable for our purposes). - **Falcon (TII)** – Falcon 7B or 40B models are fully open (Apache 2.0). Falcon 40B is powerful (on par with older GPT-3.5), but running it is heavy. Falcon 7B is lighter but might be less accurate. Still, Falcon models were trained on a lot of data and are good at factual responses. - **Mistral 7B** – A newer 7B model that **outperforms Llama-2 13B on many benchmarks** ²¹ . It's also Apache 2.0 licensed (no usage restrictions) ²² , and designed for efficiency (uses grouped-query attention for faster inference). Mistral 7B could be a great choice if you want a smaller model that punches above its weight in performance. Because it's only 7B, it can be run on CPU (with 4-bit quantization) or on a modest GPU. - **Others:** There are many variants and fine-tuned versions (e.g. Vicuna, which is LLaMA-based, or newer models like Qwen, etc.). For a finance focus, you might consider if there's a model tuned on finance data (e.g. Bloomberg released a finance model, but it might not be fully open). To keep things simple, we can start with a general model and rely on the retrieval to provide domain-specific knowledge.

For our prototype, let's assume we choose **Mistral 7B** or **Llama-2 13B Chat** depending on what our hardware can support. We'll deploy it in a way that it can handle the context length of our prompts (make sure the model supports at least 4k tokens to fit question + context; most do). We won't need to further fine-tune this LLM – we'll use it as is, since retrieval provides the necessary info. However, we will **prompt** it properly to ensure it behaves (we'll craft a prompt template for Q&A with instructions, coming up next).

Embedding Model (for Retrieval): The embedding model can be much smaller and runs quickly. Options: - **SentenceTransformers models** like `all-MiniLM-L6-v2` (384-dimensional embeddings, very fast, decent general performance) or `all-mpnet-base-v2` (768-dim, somewhat better but slower). - **E5-large (Embedding for Information Retrieval)** – a 1024-dim model known for strong retrieval performance, including on QA tasks. Might be slightly heavier to run but still feasible on CPU. - **InstructorXL** or **text-**

embedding-gecko from Hugging Face – these are other open models that sometimes rival OpenAI's embedding. We can pick one that has good multilingual/finance coverage if needed. - Since our documents and queries are in English and fairly formal text, a general embedding model should work well. For simplicity, we'll pick one of the popular ones (say MiniLM or MpNet). These are available on HuggingFace Hub and can be used via the `langchain.embeddings.HuggingFaceEmbeddings` class easily.

Why not use the LLM itself for embeddings? In some setups, the same LLM can generate embeddings (for example, using the LLM's hidden layers). However, that's not common with GPT-style models and would be inefficient. It's better to use a purpose-built, smaller model for embedding the documents and queries.

Model Hosting Considerations: As mentioned, the LLM will either run on **SageMaker** or **EC2**: - If SageMaker: We can use a pre-built container. For instance, Amazon JumpStart provides one-click deployment for popular models (Llama2, Falcon, etc.). We'd specify the model artifact (or use a pre-hosted one if available), and deploy an endpoint. For Mistral 7B, we might need to bring our own container (like using the HuggingFace Deep Learning Container and loading the model from HuggingFace Hub). This might require writing a small inference script but Amazon has examples for HF Transformers. - If EC2: We can set up an EC2 (possibly with GPU for speed). The **Deep Learning AMI** on AWS comes with PyTorch pre-installed which can be handy. We'd then `pip install langchain, transformers, etc.`, download the model weights (maybe from HuggingFace), and run our chain. We could wrap it in an API (Flask/FastAPI) that the UI can call. This is more manual but gives full control.

Since cost is a concern, one approach is to develop on an EC2 (which you can shut down when not in use to save money) and for production-like deployment, consider SageMaker's serverless inference (which only charges per request, avoiding idle charges). SageMaker Serverless has some limitations (6 GB memory limit, which might just handle a 7B model if quantized). Alternatively, a small GPU instance on SageMaker or EC2 on a schedule (turned off at night) could be used.

In either case, **the embedding model** can run on CPU fairly quickly, so it could even be part of the Lambda function or on the same host as the LLM. For example, we might load the embedding model inside the Lambda so it can embed the user query on the fly (embedding one short question is very fast, a few hundred milliseconds). Meanwhile, the document embeddings are precomputed.

To sum up, our model choices embrace open-source to keep things free and flexible. By selecting the appropriate size models, we ensure the system can run with minimal infrastructure. Next, let's build the core Q&A chain with LangChain, putting together the retrieval and generation steps.

Implementing the Retrieval QA Pipeline (LangChain Chains)

With data indexed and models chosen, we can construct the RAG pipeline using LangChain. The core logic will be: **take user question -> retrieve relevant texts -> feed into LLM -> output answer with sources**. LangChain provides abstractions to help with each part.

Here's a breakdown of the implementation steps and how LangChain components come into play:

1. Retrieval Component:

We create a **Retriever** using our vector store. For example, if we used Chroma or FAISS, we can wrap it with

`VectorStoreRetriever`. We might configure it to return the top 3-5 most similar chunks for any query. This retriever will use the embedding model to encode the incoming question and perform the similarity search. Thanks to our indexing, this is efficient – even a thousand documents can be searched in under a second or two. We should also consider using **metadata filters** if needed (e.g. restrict to certain company if the question names one, etc., which LangChain retrievers allow). But initially, top-k similarity is fine.

2. Prompt Template for Generation:

We'll craft a prompt that instructs the LLM on how to use the retrieved information. This is crucial for getting good answers. A typical prompt template might be:

```
SYSTEM: You are a helpful financial assistant. Answer the question truthfully
using the provided context from financial documents. If the answer is not in the
context, say you don't know. Do not provide advice. Cite the source of each fact
in parentheses.
```

```
USER QUESTION: {question}
```

```
CONTEXT:
{summaries_of_retrieved_documents}
```

```
ASSISTANT:
```

In LangChain, we can implement this as a `PromptTemplate` where `{question}` will be filled with the user's query, and `{summaries_of_retrieved_documents}` will be filled with a formatted string of the retrieved chunks. We may preface each chunk with something like "Document 1: [text]" so that the assistant can refer to "Document 1" in its answer citation. Simpler is to just include the text and perhaps its source title. The instructions clearly tell the model to only use that information and to include citations. This addresses the **hallucination problem** by grounding the model in provided context ²³.

3. The QA Chain:

LangChain offers a pre-built chain called `RetrievalQA` which combines a retriever and an LLM under the hood. We could use that: it will handle using the retriever to get docs and then format a prompt to the LLM. However, since we want more control (like customizing the prompt and adding citations formatting), we might instead build a custom chain. For example:

- Use `retriever.get_relevant_documents(question)` to obtain docs.
- Format the prompt manually or with a `PromptTemplate` and `LLMChain`.
- Call the LLM (which could be a `HuggingFacePipeline` or `ChatModel` wrapper in LangChain) with the prompt and get the output.
- Finally, present the answer.

We can also use `ConversationalRetrievalChain` from LangChain if we want to maintain conversation context automatically (it handles remembering previous Q&A pairs), which leads us to...

4. Memory for Conversation:

To make the assistant context-aware of the conversation, we integrate **LangChain Memory**. One simple way is to use `ConversationBufferMemory` which keeps a running log of past exchanges. We can attach this to the chain so that each new query the user asks will have the previous Q&A in context as well. This

allows follow-up questions like “What about for last year?” to be understood in context (the assistant knows “last year” refers to the previous question’s company’s revenue, for example).

Another approach is to only store the last user question and answer as context (to avoid prompt getting too large), or use a summary memory that summarizes older parts. For a first iteration, a buffer memory of say the last 2-3 interactions is fine.

When configuring memory in LangChain, we set the `return_messages=True` or use a `ChatPromptTemplate` so that it can feed the conversation history along with new question and retrieved docs to the LLM. We must ensure the retrieved docs + conversation history fit in the model’s context length.

5. Chain vs Agent:

Up to now, we described a straightforward chain: question -> retrieve -> answer. LangChain can implement this as a single chain. But the prompt we give can also be designed for an **agent**. Agents in LangChain use a **ReAct framework** (they can reason and use tools in steps). Do we need an agent here? Possibly for certain tasks: - If a question requires a calculation (e.g. “What is the percentage change in revenue between 2021 and 2022 for Company X?”), the assistant might retrieve the two numbers from documents, then perform a calculation. We could equip an agent with a simple calculator tool to do this math reliably rather than expecting the LLM to do it in-text (LLMs can do basic math, but precise calculations are safer with a tool). - If we want the assistant to answer from filings but also have the ability to fetch **live news** if asked about something very recent, we could give it a “search the web” tool. However, that would require an external API (which might break our fully open-source rule and add cost). Alternatively, if our news dataset is updated, retrieval covers it. So a web search tool might not be needed if we keep the index fresh.

To demonstrate **agents and tools**, we can set up an agent in LangChain that has access to: - A “Vector DB Search” tool (which when invoked, uses the retriever to get docs). - A “Calculator” tool (could be simply Python’s eval for math or a built-in math tool LangChain offers). - (Optionally, other tools like a knowledge base lookup, but in essence the vector search is our primary knowledge tool).

Using an agent is more complex; it means the LLM will decide when to call these tools. We would give it a prompt like: *“You are a financial assistant with access to the following tools: SearchDocs, Calculator. Use them to find information and answer the question. Always cite sources from documents. Begin!”* The agent will then go through a reasoning loop: e.g. *User asks about revenue change -> Agent decides to call SearchDocs with query “Company X 2021 revenue” -> gets doc chunk with 2021 and 2022 revenues -> Agent then calls Calculator with those numbers -> gets result -> Agent formulates final answer with the result and source.* LangChain’s agent framework can handle the sequence of calls and we can observe its reasoning via callbacks.

For our project, using an agent is an **advanced feature**. If you’re new to LangChain, you might first implement the simpler RetrievalQA chain. Once that works, you can refactor into an agent if needed for additional capabilities. The benefit of showcasing an agent is that it touches more LangChain components (tool use, planning actions) and mimics how a more general assistant might work. It’s also a good exercise in **AI integration**, showing how the AI can interact with other system components (like a calculator or database) to solve tasks.

6. Callbacks and Instrumentation:

As we build and test this pipeline, we should enable **LangChain callbacks** to monitor the process. LangChain has verbose logging and callback handlers that can show each step (retrieved docs, tokens

generated by the LLM, etc.). During development, we can use the `ConsoleCallbackHandler` (or simply set `verbose=True` on chains) to see what's happening. In production, we can route these logs to AWS CloudWatch. For example, we can implement a custom callback that writes each query and answer, plus timing info, to CloudWatch Logs or X-Ray for tracing. This is invaluable for **monitoring** once deployed – you can trace how the LLM arrived at an answer, which helps debug any bad responses or delays. LangChain even has integration with **LangSmith** (a tracing tool) for deep analysis, but to keep things open-source, we can stick to standard logging.

7. Putting it Together:

We will wrap the retrieval and generation steps into a single function that our AWS Lambda (or backend) can call for each user query. Pseudocode for the core logic might look like:

```
# Pseudocode for query handling
def answer_query(user_query, chat_history):
    # 1. Retrieve relevant docs
    docs = vector_retriever.get_relevant_documents(user_query)
    # 2. Construct the prompt with context
    context = "".join(f"- {doc.metadata['source']}:\n{doc.page_content}\n" for
doc in docs)
    prompt = prompt_template.format(question=user_query, context=context,
history=chat_history)
    # 3. Get LLM answer
    answer = llm.generate(prompt)
    return answer
```

(This is a simplified view – LangChain's chain would do similar under the hood. If using an agent, the logic differs as the agent runs its own loop.)

We should test this locally or in a notebook with sample questions to ensure it returns reasonable answers with citations. For instance, ask: "What was ACME Corp's revenue in 2022?" and see that it pulls the number from the 10-K and cites e.g. "(ACME 2022 Annual Report)". If the output isn't including citations as we like, we tweak the prompt instructions further (prompt engineering is an iterative process!).

By this point, we've built the brain of our assistant. Next, we tackle deployment – making this accessible via a web interface on AWS, and ensuring we follow MLOps best practices for reliability.

Developing the Web UI (User Interface)

To let users interact with our QA system, we'll create a simple web-based interface. The UI doesn't need to be fancy – even a basic HTML page with a text box and a submit button is enough – but it should be **user-friendly and clear** in presenting results with sources.

Option A: Static HTML/JavaScript on S3 – We can write an `index.html` that has: - An input field for the question. - A "Submit" button (or we can trigger on Enter key). - An area to display the answer (which might include rich text with citations or links). - (Optional) A conversation log if we want to show the chat history.

Using plain JavaScript (or a lightweight framework), we'll call our backend API. For example, using the Fetch API:

```
fetch("https://<api_id>.execute-api.<region>.amazonaws.com/prod/ask", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({ question: userQuestion })
}).then(response => response.json())
  .then(data => displayAnswer(data.answer));
```

We assume our API Gateway is set up to handle a POST request to an `/ask` endpoint, which triggers the Lambda that runs our LangChain query function.

When the response comes back, we will display it. We should ensure the answer text retains the citation formatting. One idea is to format citations as hyperlinks in the answer. For instance, if our answer text contains something like `(Source: 2022 10-K)`, we could post-process that in JS to link to the actual document (if we have a URL for it). This requires that we stored the document URLs in metadata and maybe returned them. Alternatively, we could include direct URLs in the answer from the start. Simpler: just show the answer and source names, and perhaps list the sources at the bottom, e.g. "Sources: 2022 10-K, Q4 2023 Earnings Call". The user can then look them up (or we provide a hyperlink to the document in S3 or an external site).

For a nicer UI, one could use a library like **Gradio** or **Streamlit** to make a quick chat interface. Gradio, for example, can easily create a text chat widget and can be hosted with minimal fuss. However, hosting Gradio on AWS would mean running a server (which could be done on EC2). Since we want to leverage AWS services like S3 and keep costs low, a static site is more cost-effective (no server for UI needed).

We will: - Upload `index.html` (and any CSS/JS files) to an S3 bucket. - Enable static website hosting on that bucket (and if needed, set the policy public or use CloudFront for a nicer domain). - Ensure our API Gateway has CORS enabled so the JS can call it from the domain of the static site.

The UI should also handle basic UX points: - Show a loading indicator while the question is being answered (since it could take a few seconds for the LLM to respond). - Possibly allow the user to ask another question continuing the conversation. We might maintain the `chat_history` in the browser (or rely on the backend to manage session state via a user ID or something). For simplicity, we could send the entire conversation each time (though that might become large). - Display the answer in a formatted way. We could highlight the source citations (maybe italicize them or put them in a smaller font). - If the backend returns an error or "I don't know", display that gracefully.

Example UI flow: The user goes to the webpage, enters "What is ACME's gross profit margin for 2023?" and hits submit. The page calls the API, and after a short wait, the answer appears: "**ACME's gross profit margin in 2023 was 35%**, according to the 2023 annual report ²⁴ ." and perhaps a line "Source: ACME 2023 10-K (pg 45)". The user can then ask "How does that compare to 2022?" and the assistant will use memory + retrieval to answer the follow-up.

Though simple, this UI fulfills the requirement of a “simple but effective web-based UI.” It allows anyone to query the assistant easily. In a real scenario, further UX polish could include user authentication (if it's an internal tool), or multiple tabs for different companies, etc., but we'll keep it minimal here.

Now that we have the frontend, let's detail how to deploy our backend on AWS and tie everything together, along with our plans for CI/CD and monitoring.

Deployment on AWS (Backend Setup and Integration)

Deploying our solution involves setting up the AWS services and making sure they work together. We have several components to configure: S3, Lambda, API Gateway, and possibly SageMaker or EC2 for the model. Let's go step-by-step:

1. AWS Environment Setup:

Make sure you have an AWS account with the necessary permissions to create the resources (S3, Lambda, API Gateway, SageMaker/EC2). Also, configure your AWS CLI or SDK credentials if you plan to deploy via scripts. For CI/CD, you might use Infrastructure as Code (IaC) tools like AWS CloudFormation or Terraform, but you can also start by manually creating resources in the AWS Console for a proof of concept.

2. Create an S3 Bucket for Data and UI:

- Choose an S3 bucket name (must be globally unique). For example, `finance-rag-project-<yourname>`. - Upload your **document files** to a folder in this bucket (e.g. `documents/CompanyX_10K_2023.txt` etc.). You can organize by company or date as needed. These could also remain local if you already embedded them, but having them on S3 means you have a source of truth storage. - Upload your **front-end files** (HTML/JS/CSS) to the bucket, perhaps under `site/`. Enable static website hosting on the bucket (set the index document to your HTML file). If you want the site public, adjust the bucket policy accordingly (or use CloudFront with an OAI for security – optional detail). - (Optional) If your vector index is a file (like `index.faiss` or a Chroma SQLite DB), you could store that on S3 as well so that your Lambda/EC2 can load it. Alternatively, you'll generate the index on the fly in the backend initialization.

3. Deploying the LLM Model:

We have two main approaches here:

A. Using Amazon SageMaker:

- Package the model. If SageMaker JumpStart offers the model (e.g. “Llama-2 13B-chat”), you can deploy it via the console by selecting it and choosing deployment configuration. If not, you can use the HuggingFace Deep Learning Container. For example, use the AWS CLI or SDK to create a model pointing to the HuggingFace image `huggingface-pytorch-linux-x86_64` and set the model data to the S3 path of your model weights (you might need to upload the model to S3 if you downloaded it locally). - Create a SageMaker endpoint configuration with instance type (say `ml.g5.xlarge` for a 1xA10 GPU, or `ml.g4dn.xlarge` for T4 GPU, or even `ml.m5.2xlarge` for CPU if quantized). - Create the endpoint. This will spin up the instance and load the model. Note the endpoint name. - Test the endpoint (you can invoke it via boto3's `invoke_endpoint` to ensure it returns something for a test prompt). - SageMaker handles scaling (you can configure auto-scaling if needed) and will output logs to CloudWatch.

B. Using an EC2 Instance:

- Launch an EC2 (choose an AMI with your desired OS; if you need a GPU, pick a GPU instance type and an NVIDIA CUDA AMI or Deep Learning AMI). - SSH into the instance, install required libraries (Python packages like transformers, langchain, etc.). Also transfer your model weights to the instance (either by downloading from HuggingFace or copying from S3). - Run your server code. For instance, you might run a Flask app that listens for POST requests at `/ask`. Inside that endpoint, you call the LangChain chain to get an answer. Ensure this Flask app is launched (maybe via gunicorn for production). - Security: open the necessary port (e.g. 8000) in the EC2 security group, but if exposing directly, be cautious (maybe only allow your IP for now). If using API Gateway in front, the EC2 could even be in a private subnet and API Gateway calls it via a VPC link. - This approach might be simpler for initial development (just run the Python app and call it), but for production you'd want to daemonize the process and ensure the instance is secure.

We will proceed assuming **Approach A (SageMaker)** for the model, as it uses AWS managed services and aligns with "serverless" theme.

4. Building the Lambda Function:

This Lambda will orchestrate the RAG process: - It will receive API requests (with the user's question, and possibly a conversation ID or session history). - It will use the vector store to retrieve relevant docs. - It will call the SageMaker endpoint with the composed prompt (including context). - It will get the response, format it (perhaps ensuring it's JSON serializable), and return it via API Gateway.

To implement this, our Lambda needs the LangChain code and possibly the embedding model: - Use a Lambda container image or a deployment package. Since LangChain and Transformers can be large, the container approach is convenient (AWS Lambda supports container images up to 10 GB). We can base it on a Python image, install `langchain`, `boto3`, `faiss` (if using), etc., and copy any necessary files (like a saved FAISS index). - Alternatively, we could try a non-container Lambda deployment by trimming dependencies, but it might be tricky to fit within the 250MB limit uncompressed. So let's assume a container. - Give the Lambda an IAM role with permission to read from S3 (to load data or index), and to invoke the SageMaker endpoint (`sagemaker:InvokeEndpoint` permission). - In the Lambda code (which could be a Flask app converted to work in Lambda via AWS API Gateway's integration, or simply a handler function), do something like:

```
import os, json, boto3
from langchain.vectorstores import FAISS
from langchain.embeddings import HuggingFaceEmbeddings
# possibly load FAISS index from file or S3 in global scope for reuse across
invocations
faiss_index = FAISS.load_local("/tmp/index.faiss",
                               embeddings=HuggingFaceEmbeddings(...))

sm_runtime = boto3.client('sagemaker-runtime')

def handler(event, context):
    question = json.loads(event['body'])['question']
    # Get relevant docs
    docs = faiss_index.similarity_search(question, k=3)
```

```

context_text = ""
for i, doc in enumerate(docs, 1):
    source = doc.metadata.get('source', f"Doc{i}")
    context_text += f"{source}: {doc.page_content}\n"
prompt = PROMPT_TEMPLATE.format(question=question, context=context_text)
# Call LLM via SageMaker
response =
sm_runtime.invoke_endpoint(EndpointName=os.environ['SM_ENDPOINT'],
                           ContentType='application/json',
                           Body=json.dumps({"inputs": prompt}))
result = json.loads(response['Body'].read().decode())
answer_text = result[0]['generated_text'] if isinstance(result, list) else
result
# Return answer
return {
    "statusCode": 200,
    "headers": {"Content-Type": "application/json"},
    "body": json.dumps({"answer": answer_text})
}

```

This is a simplified sketch. The actual code will depend on the model's serving format. For instance, some HF model endpoints expect `{"inputs": prompt}` and return a JSON list with `generated_text`. We'd adjust accordingly. Also, we might need to handle the conversation history by storing it in the event of multi-turn chat (could use an in-memory dict or DynamoDB for session storage, or pass it from front-end). - After writing the code, build and push the Lambda container (if using that) to ECR, then create the Lambda function from that image.

5. Setting up API Gateway:

- Create a new REST API (or HTTP API) in API Gateway.
- Define a resource/endpoint, e.g. `/ask` with POST method.
- Integrate it with the Lambda function we just created (use Lambda Proxy integration to easily forward the request).
- Enable CORS on the endpoint so that your S3 web frontend (which will likely be on a different domain) can call it.
- Deploy the API to a stage (say "prod") and note the invoke URL.

6. Testing End-to-End:

Now we test the whole flow:

- Open the web UI (the S3 site URL). Enter a sample question.
- The browser calls API Gateway, triggers Lambda.
- Lambda retrieves context and calls SageMaker (or the EC2) and gets answer.
- The answer returns through API Gateway to the browser and is displayed.
- Check that the answer makes sense and includes sources.
- Monitor logs: in CloudWatch, the Lambda logs should show the steps taken. The SageMaker endpoint will also have logs (in CloudWatch Logs for the container) where you might see any errors or invocation times. This is where our **callbacks** can be useful – we could have the Lambda print out which documents were retrieved and how long the LLM took, etc.

7. Optimizing and Troubleshooting:

If the latency is high, consider:

- Reduce number of documents retrieved or prompt size.
- If using SageMaker, ensure the instance isn't memory-swapping (check endpoint logs). Maybe a larger instance or a smaller model or quantization could help.
- Lambda cold start: keep an eye on it. Using Provisioned Concurrency for Lambda can help if constant usage is needed (though it has cost).
- If memory is an issue in

Lambda (loading too many libs), ensure the container has only what's needed. Possibly do heavy loading (like reading the index from S3) outside the handler so it stays loaded between calls.

Cost considerations: By using Lambda+API Gateway, we pay per request (fractions of a cent per invocation plus data transfer). S3 costs are negligible for our data sizes. The main cost is the SageMaker or EC2 for the model: - If SageMaker on a GPU instance, that could be ~\$0.5 per hour. If usage is only a few times a day, it's wasteful to run it constantly. You can script the endpoint to spin down (delete) when not needed, and spin up on demand (but spin-up might take a minute or two to load model). - One idea: use SageMaker **serverless**. It charges per invocation time, and automatically scales to zero when idle. If our model fits (7B might fit if optimized), this could drastically cut costs for low usage, since you only pay for actual usage time. - If using EC2, you can shut the instance down after use (and start when needed). Or use an **EC2 Spot instance** if acceptable, to reduce cost ~70%. But spot can be terminated anytime, so not ideal for a persistent service unless you handle interruptions. - Monitor your AWS billing and adjust resources accordingly. This is part of the **MLOps responsibility** – to ensure the solution is cost-effective and adjust when it's not.

At this point, we have a working deployed system. The remaining tasks are around **MLOps best practices** – making sure we can maintain and improve this system over time.

CI/CD and MLOps Best Practices

Building the system is half the battle; maintaining and improving it is the other half. Here we outline how to apply **CI/CD (Continuous Integration/Continuous Deployment)** and other MLOps practices to our project:

Version Control and CI:

All our code (for data processing, the LangChain pipeline, the Lambda function, the UI, etc.) should live in a version-controlled repository (e.g. GitHub or GitLab). This includes infrastructure configuration if using IaC. We will write **unit tests** for as many components as makes sense: - For example, a test for the retrieval function (given a sample question, does it return chunks that contain the answer?). - A test for the prompt formatting (ensuring that if we feed a known context and question, the prompt is constructed as expected). - If possible, an integration test using a smaller dummy model locally to simulate the whole chain.

Using a CI service (like GitHub Actions), we can run these tests on each commit. This ensures we catch any breaking changes early (important if multiple collaborators, or even for yourself to avoid accidentally breaking the pipeline with a code change).

Automated Deployment:

We can incorporate deployment steps into our CI pipeline: - For the Lambda: build and push the container automatically when code changes. For example, a GitHub Action can build the Docker image and push to AWS ECR, then update the Lambda function. - For the UI: similarly, if the front-end changes, we can have the pipeline upload the new `index.html` to S3 (maybe using AWS CLI or an S3 GitHub Action). - For infrastructure (API Gateway, SageMaker, etc.): if using IaC like a CloudFormation template or Terraform config, that can be applied via CI too. If not, at least document the manual steps or use AWS SAM for the Lambda+API setup. - Each deployment should be versioned. Tag releases (v1.0, v1.1, etc.), especially when you update the model or data.

Model Versioning:

Keep track of which model (and which version of it) you are using in production. For instance, if you start with “Llama-2 13B chat v1”, record that. If later you switch to “Mistral 7B v0.1”, note that as a new version. You can use tags or even a model registry. Because we’re using open models, updating might involve just changing the HuggingFace model reference. Always test new models thoroughly (they might have different quirks). If you fine-tune or customize a model, store those weight files systematically (e.g. in S3 with a version identifier).

Data Versioning and Retraining:

In RAG, “retraining” usually means updating the document index. We should establish a process for this: - For example, every quarter new 10-Q filings come out. We should fetch those and update the vector store. This could be a scheduled job (AWS EventBridge triggering a Lambda to ingest new data, or a manual process if fine). When new data is added, we might also remove or archive stale data (or keep it if needed for historical questions). - If we significantly change the data (like add a whole new set of companies), consider re-evaluating the embedding model choice or any prompt adjustments. - Maintain a **changelog** of data updates (e.g. “Added Q4 2025 transcripts on Jan 30, 2026”).

Monitoring and Logging:

We want to monitor: - **Usage**: How many queries per day? What questions are being asked? (This helps understand user behavior and maybe identify needs for more data or features. Just be mindful of PII – in finance case, probably fine.) - **Latency**: Track the end-to-end latency for each query. CloudWatch Logs can show Lambda duration, and SageMaker endpoints have invocation metrics (like Invocations count, latency). Set up CloudWatch Alarms if latency spikes or errors occur frequently. - **Errors**: Ensure any exceptions in Lambda are logged. Also watch for LLM errors or timeouts. Sometimes the model might not handle a prompt and could crash; we need to catch that, maybe by wrapping the call in a try/except and returning a user-friendly error (“Sorry, I couldn’t process that request.”). - **Quality**: This is trickier to auto-monitor, but we can periodically sample some interactions and review them. If users can give feedback (like thumbs up/down on answers), log that. This ties into **Quality and Risk Management** – if the assistant is frequently wrong about certain things, we need to address it (maybe add more context, adjust the prompt, or in worst case, fine-tune the model). - **Model drift**: Because the model is static until changed, “drift” in the traditional sense is not an issue, but drift in data is – which we handle by updating the index. If the financial world changes (new terms, new regulations), ensure the system is updated to handle those (could be solved by adding those topics into context or upgrading the model if needed). - Use AWS CloudWatch dashboards to visualize key metrics (invocations, errors, latency, SageMaker CPU/memory if possible).

Alerting:

As part of monitoring, set up alerts: e.g. if the Lambda errors out 5 times in a row, send an email or Slack message to the developer. If the SageMaker cost in a day exceeds X (to catch runaway usage), alert it. This ensures quick response to issues, important for production stability.

Maintenance and Continuous Improvement:

Regularly plan to: - Incorporate user feedback and new requirements. Maybe stakeholders now want the assistant to also handle some analytical tasks (like “compare company A and B”). You could then plan to implement that (maybe by enhancing the agent to retrieve from multiple company docs and then summarizing). - Stay updated with latest **AI research/tech**: New models or tools may emerge that can improve your system. For example, if a new open-source model comes that’s much better at financial QA, you might integrate it (thanks to our modular design, we can swap models easily ¹⁷). - Manage technical

debt: if some parts of the code were quick hacks, refactor them. Keep documentation up to date so that if you (or someone else) revisit the project in 6 months, it's clear how everything is configured. - Ensure **best practices** are followed: code is clean, secrets (like API keys or AWS credentials) are handled securely (never hard-coded; use AWS Secrets Manager or environment variables for any sensitive info), and the system is scalable for larger workloads if needed (maybe consider multi-threading or batching in retrieval if expecting heavy usage).

In summary, treating this project with a production mindset from the start will enforce good habits. You've effectively demonstrated **technical leadership** by designing a robust, maintainable system and outlining how to keep it running optimally.

Finally, let's tie everything together with an example and conclude.

Example Usage and Results (Demonstration)

To illustrate the end result, here's a hypothetical interaction and how the system handles it:

- **User question:** "What was ACME Corporation's net income in 2022, and what did the CEO say about it in the earnings call?"
- **System process:** The query is received by Lambda via API Gateway. The vector store is queried with the embedding of this question. Likely, it finds:
 - A chunk from *ACME 2022 Annual Report* that mentions net income for 2022 (say, "Net income for 2022 was \$5.4 million, a 10% increase from 2021...").
 - A chunk from *ACME Q4 2022 Earnings Call Transcript* where the CEO or CFO discusses the yearly results (maybe "CEO: We are pleased with our net income growth, driven by cost efficiencies...").
 - Perhaps one more related chunk (or if those two suffice, just them).
- These texts and their sources are inserted into the prompt template. The prompt might look like:

```
You are a financial assistant...
Question: What was ACME Corporation's net income in 2022, and what did the
CEO say about it in the earnings call?
Context:
ACME 2022 Annual Report: "... Net income for 2022 was $5.4 million, a 10%
increase from 2021 ..."
ACME Q4 2022 Earnings Call: "CEO: 'We are pleased with our net income
growth... it shows our strategy is working ...'"
```

- **LLM generation:** The model sees this context and question, and produces an answer along the lines of: *"ACME Corp's net income in 2022 was \$5.4 million, which was about 10% higher than the previous year. In the Q4 2022 earnings call, the CEO expressed satisfaction with this growth, stating that it 'shows our strategy is working' and credited cost efficiencies for the improved profit ²⁴."* (Here ²⁴ would correspond to a citation from the context; in practice, our assistant might cite as (2022 Annual Report) and (Q4 2022 Call) rather than a numeric citation since we aren't literally using those brackets format in prompt, but the idea is it references the sources.)

- **Output to user:** The front-end displays the answer, nicely formatted, and lists **Sources:** “2022 Annual Report, Q4 2022 Earnings Call Transcript” (each could be hyperlinked to the document if we have the URLs).

This answer is fact-based and traceable to the source, fulfilling the user’s query with the expected context and attribution. The user can follow up with something like “How does that compare to 2021?” and because we have conversation memory, the system knows we are still talking about net income and ACME, and can answer accordingly using the 2021 data from the context (maybe retrieving that chunk now).

Throughout this process, our logging captured the steps, and if something went wrong (say the model gave an irrelevant answer), we would investigate by checking the retrieved docs and the prompt, adjusting as needed (e.g. add a system instruction “Don’t answer if not in context”).

Conclusion and Key Takeaways

We have now built a comprehensive RAG-based AI assistant in the finance domain, covering the entire journey from concept to deployment. This project integrates all key aspects of AI engineering:

- **Stakeholder Collaboration:** We began by understanding the needs (accurate financial answers with sources, no unwanted behavior). This ensured our technical solution stayed aligned with business goals.
- **Technical Design & Leadership:** We architected a modular system using open-source tools, making conscious choices about models, data stores, and AWS services to balance performance and cost. The architecture diagrams and rationale provided show the thinking process behind each decision.
- **AI Integration:** We didn’t build an AI in isolation – we integrated it into a full application stack (with a UI, database, APIs) and into the specific *financial domain context*. The assistant augments human analysts’ capabilities by quickly retrieving and summarizing information from piles of documents.
- **Use of LangChain Components:** We utilized chains for the Q&A workflow, memory for context carry-over, tools/agents for extended functionality, prompt templates for consistent LLM instructions, and callbacks for tracing. This thorough use of LangChain demonstrates how to leverage such a framework to build complex applications efficiently.
- **MLOps, Monitoring & Maintenance:** We outlined how to deploy the system on cloud infrastructure, automate the process, and monitor its health. Logging, versioning, and test pipelines ensure that as we update the system (new data or model), we can do so reliably. We also have a plan for continuously updating the knowledge base (e.g. new filings) so the assistant remains current ²⁰ . Alerts and metrics help maintain **accountability** – if something breaks or drifts, we will know and can fix it proactively.
- **Financial Domain Focus:** Throughout, we paid attention to domain-specific considerations: precision of financial figures, the need for source verification, avoiding advice, and using proper terminology. For instance, we included actual filings and transcripts to ground the model, because finance is an area where factual accuracy is crucial. This shows an understanding of applying AI to a specific industry with its unique requirements.

By following this guide, a single Python developer can not only build a powerful RAG system, but also learn **end-to-end AI project development**. The steps provide a template for tackling similar projects in other domains as well. Remember to iterate in small steps (load a couple of documents and test retrieval before scaling up, test the LLM prompt on a simple question before deploying, etc.). Each component we built can be tested and improved independently, which is a sound engineering practice.

Finally, this project showcases how modern open-source AI (LLMs like Llama-2/Mistral and frameworks like LangChain) combined with cloud services can create **practical solutions** that were almost unimaginable just a couple of years ago. By mastering this, you'll be well-equipped as an **AI Engineer** who can deliver real value – finding insights in financial data swiftly, accurately, and transparently, all while maintaining the systems and collaborating effectively with stakeholders.

Good luck with building your Finance QA assistant, and enjoy the learning process!

Sources:

1. LangChain Documentation – *Retrieval-Augmented Generation Tutorial* ¹⁰ ¹⁹ (Overview of RAG pipeline indexing and retrieval steps)
2. AWS Public Sector Blog – *Modular RAG Architecture on AWS* ¹³ ¹⁴ (Reference architecture using Lambda, API Gateway, vector store, SageMaker for RAG)
3. AWS “What is RAG?” – *Retrieval-Augmented Generation Explained* ¹ ³ (Benefits of RAG for accuracy and source attribution)
4. Medium (Rishabh Singh) – *Financial Analysis Chatbot with RAG* ²⁴ (Using SEC filings from EDGAR as data source for a finance chatbot)
5. Scribble Data Blog – *Top 10 Open-Source LLMs 2025* ²¹ ²² (Open LLM choices like Mistral 7B performance and licensing)

¹ ³ ²⁰ ²³ What is RAG? - Retrieval-Augmented Generation AI Explained - AWS

<https://aws.amazon.com/what-is/retrieval-augmented-generation/>

² ⁵ ⁶ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ Use modular architecture for flexible and extensible RAG-based generative AI solutions | AWS Public Sector Blog

<https://aws.amazon.com/blogs/publicsector/use-modular-architecture-for-flexible-and-extensible-rag-based-generative-ai-solutions/>

⁴ ²¹ ²² The Top 10 Open Source LLMs: 2025 Edition - Scribble Data

<https://www.scribbledata.io/blog/the-top-10-open-source-llms-2024-edition/>

⁷ ⁸ ⁹ ¹⁸ ²⁴ Financial Analysis Chatbot for 10-Q/10-K Reports Using Retrieval-Augmented Generation (RAG) | by Rishabh Singh | Medium

<https://medium.com/@RobuRishabh/financial-analysis-chatbot-for-10-q-10-k-reports-using-retrieval-augmented-generation-rag-ef3938892086>

¹⁰ ¹⁹ Build a Retrieval Augmented Generation (RAG) App: Part 1 | LangChain

<https://python.langchain.com/docs/tutorials/rag/>